

Hackathon BiT - Equipa 48 - Arquitectura

1. Objectivo do documento

Este documento define a arquitectura final do projecto, com foco em:

- clarificar o que será construído no MVP;
- distribuir responsabilidades por membro da equipa;
- alinhar as prioridades para as próximas semanas;
- garantir uma base técnica escalável, simples de manter e fácil de evoluir;
- evitar dispersão de esforço em funcionalidades que não aumentem a qualidade da demonstração final.

A ideia central do produto é transformar dados públicos e o dataset Visent CDRView numa ferramenta de decisão para gestores públicos, com mapa, indicadores cruzados e consultas em linguagem natural.

2. Princípios da arquitectura

A solução será desenhada com estes princípios:

1. **MVP-first** — entregar primeiro o fluxo mais valioso e demonstrável.
 2. **Monólito modular** — evitar microserviços para reduzir complexidade e acelerar desenvolvimento.
 3. **Separação clara de responsabilidades** — frontend, backend, dados e IA com contratos definidos.
 4. **Dados como base do produto** — a qualidade do modelo de dados é crítica.
 5. **Escalabilidade evolutiva** — a arquitectura deve permitir adicionar novas fontes, novos indicadores e novos módulos sem reescrever tudo.
 6. **API contract-first** — frontend e backend devem trabalhar com formatos bem definidos desde o início.
 7. **IA como camada de interpretação** — a IA deve ajudar a consultar e explicar dados, não substituir a lógica do sistema.
 8. **Simplicidade operacional** — fácil de desenvolver, testar, debugar e colocar em produção.
-

3. Stack recomendada

Frontend

- **React**
- UI responsiva para desktop e mobile
- Mapa interactivo
- Tela de consulta em linguagem natural
- Tela de resultados e relatórios

Backend principal

- **Spring Boot**
- API central da aplicação

- Agregação de dados
- Gestão de indicadores
- Exposição dos endpoints para frontend
- Integração com o serviço Python

Serviço de dados e IA

- **Python**
- ETL / ingestão / limpeza de dados
- Normalização de bases
- Cálculo de agregações
- Processamento da consulta em linguagem natural
- Integração com o modelo de IA

Base de dados

- **PostgreSQL**
- Estrutura relacional para regiões, indicadores, observações e fontes
- Preparada para novas fontes e novas métricas

Deploy

- **Railway** ou **Render** para MVP
- Separação entre frontend, backend e serviço Python, mesmo que todos vivam num único repositório (a princípio)

4. Visão geral da arquitectura

```
Fontes de dados
(Vísent CDRView, datasets públicos, dados complementares)
  |
  v
Serviço Python
(ingestão, limpeza, normalização, IA)
  |
  v
PostgreSQL
(modelo relacional e dados tratados)
  |
  v
Backend Spring Boot
(API principal, regras de negócio, agregações)
  |
  v
Frontend React
(mapa, consulta, dashboards, relatórios)
```

Leitura da arquitectura

- O **Python** trata o que for mais rápido e natural para dados e IA.
 - O **PostgreSQL** mantém os dados organizados e consultáveis.
 - O **Spring Boot** actua como orquestrador do produto.
 - O **React** apresenta os dados de forma simples e convincente.
-

5. Arquitectura de dados

A arquitectura de dados deve suportar o cruzamento entre:

- concentração de pessoas;
- cobertura de rede;
- emprego;
- formação;
- mentoria;
- saúde mental;
- iniciativas comunitárias.

5.1. Camadas de dados

a) Raw Layer

Dados tal como foram recebidos.

Objectivo:

- manter rastreabilidade;
- permitir reprocessamento;
- evitar perda de informação original.

b) Clean Layer

Dados limpos, padronizados e validados.

Objectivo:

- nomes de regiões normalizados;
- lat/lng consistentes;
- valores inválidos corrigidos ou removidos;
- colunas padronizadas.

c) Business Layer

Dados prontos para consumo pelas funcionalidades do produto.

Objectivo:

- consultas rápidas;
- visualização no mapa;
- cruzamento de indicadores;
- respostas da IA.

6. Modelo de dados sugerido

regions

Representa regiões geográficas.

Campos principais:

- `id`
- `name`
- `country`
- `lat`
- `lng`
- `metadata`

sources

Representa as fontes de dados.

Campos principais:

- `id`
- `name`
- `type`
- `url`
- `reliability_score`
- `created_at`

indicators

Representa os tipos de indicadores disponíveis.

Campos principais:

- `id`
- `name`
- `category`
- `unit`
- `description`
- `source_type`

Categorias possíveis:

- conectividade
- emprego
- formação
- mentoria
- saúde mental
- estrutura social

observations

Tabela central do sistema.

Campos principais:

- `id`
- `region_id`
- `indicator_id`
- `source_id`
- `date`
- `value`
- `unit`
- `metadata`

coverage_points

Dados geoespaciais ligados à cobertura de rede.

Campos principais:

- `id`
- `region_id`
- `technology`
- `lat`
- `lng`
- `signal_quality`
- `source_id`

programs

Programas públicos, formação, mentoria e iniciativas sociais.

Campos principais:

- `id`
- `name`
- `category`
- `region_id`
- `description`
- `status`
- `source_id`

query_logs

Registo das consultas feitas à IA.

Campos principais:

- `id`
- `user_query`

- `normalized_query`
 - `filters`
 - `result_summary`
 - `created_at`
-

7. Módulos do sistema

7.1. Ingestão e processamento de dados

Responsável por ler o dataset principal e eventuais fontes extras.

Funções:

- carregar CSV, JSON ou Excel;
- validar colunas;
- converter tipos;
- normalizar regiões;
- gravar dados no PostgreSQL.

7.2. Motor de indicadores

Responsável por expor os dados já tratados em formatos úteis.

Funções:

- filtrar por região;
- filtrar por indicador;
- calcular agregações simples;
- preparar dados para mapa e dashboard.

7.3. Serviço de IA

Responsável por converter a linguagem natural em consultas e respostas estruturadas.

Funções:

- interpretar a intenção da pergunta;
- identificar região, indicador e contexto;
- sugerir filtros;
- gerar resposta textual curta e clara;
- devolver estrutura de apoio para frontend.

7.4. Backend principal

Responsável por concentrar a lógica do produto.

Funções:

- gerir endpoints;
- validar requests;
- chamar o serviço Python quando necessário;

- consultar o banco;
- devolver dados ao frontend.

7.5. Frontend

Responsável por mostrar o valor do projecto.

Funções:

- mapa interactivo;
- tela de consulta;
- dashboard com indicadores;
- exportação visual dos resultados;
- boa experiência em mobile e desktop.

8. Endpoints principais

POST /dados

Recebe a consulta do utilizador e filtros opcionais.

Exemplo de request:

```
{
  "consulta": "Onde faltam programas de formação para jovens de baixa
renda?",
  "filtros": {
    "regiao": "Luanda",
    "indicador": "formacao"
  },
  "idioma": "pt"
}
```

Exemplo de response:

```
{
  "resposta_ia": "A região X apresenta baixa cobertura de formação e baixa
conectividade, sendo uma prioridade de intervenção.",
  "dados": [
    {
      "regiao": "Região X",
      "valor": 72,
      "fonte": "Vísent CDRView"
    }
  ],
  "fontes": ["Vísent CDRView", "Fonte pública complementar"]
}
```

GET /mapa

Devolve os dados para visualização geográfica.

Exemplo de response:

```
{
  "regioes": [
    {
      "regiao": "Região X",
      "lat": 0,
      "lng": 0,
      "concentracao": 80,
      "cobertura_rede": 35,
      "indicadores": ["emprego", "formacao"]
    }
  ]
}
```

GET /indicadores

Lista os indicadores disponíveis.

GET /regioes

Lista as regiões carregadas no sistema.

POST /ingestao/reprocessar

Reexecuta a ingestão, útil para actualizações ou correções.

9. Estratégia de escalabilidade

A arquitectura está preparada para crescer sem perder simplicidade.

O que permite escalar

- adicionar novos datasets sem trocar o modelo inteiro;
- introduzir novos indicadores;
- incorporar novas regiões;
- ligar novas fontes públicas;
- melhorar o motor de IA sem mexer no frontend;
- substituir a lógica de consulta sem alterar o contrato da API.

Como a escalabilidade foi pensada

- modelo relacional com tabelas normalizadas;
- serviço Python isolado para dados/IA;
- backend principal como ponto de integração;

- contratos JSON estáveis;
 - separação entre raw, clean e business layer.
-

10. Prioridades do MVP

O MVP deve focar o essencial.

Prioridade 1 — obrigatório

- ingestão funcional do dataset principal;
- base de dados com modelo definido;
- endpoint `/dados` funcional;
- mapa com pelo menos 2 regiões;
- consulta em linguagem natural com resposta coerente;
- frontend responsivo;
- README e instruções de execução.

Prioridade 2 — importante

- cruzamento de conectividade com emprego ou formação;
- filtros por região e indicador;
- melhoria do texto da IA;
- visualizações mais claras no mapa e dashboard;
- logs básicos de consultas.

Prioridade 3 — opcional se houver tempo

- exportação PDF;
 - alertas por limiar;
 - suportar múltiplos idiomas;
 - comparações entre regiões;
 - adicionar mais fontes públicas.
-

11. O que não deve consumir tempo no MVP

Para proteger o prazo, não entrar em:

- microserviços;
 - autenticação completa;
 - permissões avançadas;
 - sistema de contas;
 - multi-tenancy;
 - motor de relatórios demasiado complexo;
 - IA sofisticada demais;
 - integração com muitas fontes ao mesmo tempo;
 - pipeline de dados industrial.
-

12. Papéis e responsabilidades da equipa

12.1. Barwel - Project Manager

Responsável por manter o projecto coeso.

Responsabilidades

- garantir alinhamento do escopo;
- organizar as reuniões;
- acompanhar prazos e entregas;
- remover bloqueios;
- consolidar a narrativa da apresentação;
- coordenar a documentação.

Entregáveis principais

- cronograma semanal;
 - lista de prioridades;
 - acompanhamento de tarefas;
 - guião da apresentação final.
-

12.2. Paufer — Frontend principal

Responsável pela interface do utilizador.

Responsabilidades

- construir a tela de consulta;
- construir o mapa interactivo;
- montar o dashboard principal;
- integrar com os endpoints;
- garantir responsividade;
- preparar a experiência da demo.

Entregáveis principais

- interface funcional;
 - mapa com regiões;
 - resultados visuais coerentes;
 - navegação simples e rápida.
-

12.3. Paulo - Arquitectura Geral e Dados

Responsável pela estrutura de dados, integração e suporte à arquitectura geral.

Responsabilidades

- definir o modelo relacional;
- desenhar as tabelas e relações;
- organizar a ingestão do dataset;
- garantir a normalização dos dados;
- alinhar o contrato entre backend, IA e frontend;
- validar se os dados suportam os casos de uso;
- apoiar a arquitetura geral do sistema;
- manter a coerência entre problema, dados e solução.

Entregáveis principais

- esquema da base de dados;
 - pipeline de ingestão;
 - contrato de dados;
 - documentação da arquitetura;
 - validação dos indicadores prioritários.
-

12.4. Victor - Backend Developer

Responsável pela API principal e regras de negócio.

Responsabilidades

- criar o backend em Spring Boot;
- implementar os endpoints principais;
- integrar PostgreSQL;
- orquestrar chamadas ao serviço Python;
- centralizar a validação de requests;
- preparar responses consistentes para o frontend.

Entregáveis principais

- API funcional;
 - endpoints `/dados`, `/mapa`, `/indicadores`, `/regioes`;
 - integração com dados tratados;
 - base para futuras expansões.
-

12.5. Hércules - AI Engineer

Responsável pela camada de inteligência e linguagem natural.

Responsabilidades

- desenhar prompts e fluxos de interpretação;
- estruturar a resposta da IA;
- ligar o modelo ao backend ou ao serviço Python;
- definir como a IA identifica intenção, região e indicador;
- testar qualidade e consistência das respostas;

- evitar respostas vagas ou inventadas.

Entregáveis principais

- fluxo de consulta em linguagem natural;
 - prompts e templates de resposta;
 - exemplos de perguntas e respostas;
 - integração estável com os dados.
-

13. Plano para as próximas semanas

Semana 1 — fundação

Objectivo: decidir e preparar a base.

- explorar o dataset Vísent;
- fechar o modelo de dados;
- criar o repositório e a estrutura do projecto;
- montar backend e frontend base;
- definir o contrato JSON;
- fazer uma versão mockada do /dados;
- criar um mapa estático ou semi-estático.

Semana 2 — integração real

Objectivo: ligar os componentes.

- implementar ingestão real;
- popular o PostgreSQL;
- ligar backend aos dados reais;
- integrar a IA com as consultas;
- construir o fluxo de mapa + consulta;
- validar o comportamento com casos reais.

Semana 3 — acabamento e apresentação

Objectivo: transformar em demo forte.

- polir UI;
 - ajustar textos e respostas;
 - melhorar as visualizações;
 - adicionar exportação ou extras leves, se possível;
 - preparar README;
 - preparar pitch final;
 - ensaiar a demonstração.
-

14. Se sobrar tempo

Só depois do MVP sólido, considerar:

- exportação em PDF;
- filtros avançados por período;
- suporte PT / ES / EN;
- mais fontes públicas;
- alertas automáticos;
- comparação entre regiões;
- melhorias na qualidade semântica da IA;
- refinamento das visualizações geoespaciais.

A regra é simples: **não sacrificar o núcleo funcional por extras.**

15. Critério de sucesso do projecto

O projecto será bem-sucedido se, na apresentação final, for possível demonstrar claramente que:

1. os dados foram organizados com rigor;
 2. o mapa mostra desigualdades de forma visual;
 3. a IA permite consultar os dados sem SQL;
 4. a solução ajuda a priorizar políticas públicas;
 5. a arquitectura está preparada para crescer.
-

16. Resumo executivo

Este projecto deve ser tratado como uma plataforma de decisão pública baseada em dados, com:

- **React** no frontend;
- **Spring Boot** no backend;
- **Python** para ingestão, tratamento de dados e IA;
- **PostgreSQL** como base central;
- uma arquitectura modular, simples e escalável.

O sucesso não depende de fazer tudo. Depende de fazer bem o que realmente prova o valor do projecto.